```
#   { name: 'freddy', number: '555-5554', relationship: 'friend' }
# ]
```

But you don’t have to evaluate it immediately. You might want to call it later or pass it to another function. This is *lazy* because the computation is defined early, but the evaluation occurs only when `evaluate` is invoked. This is very different from just determining the computation later on.

Just like a function, this event stream filter is now first-class. For example, one of your work colleagues wants to sync live with your business contacts. You can easily send him a filtered event stream:

```
withResultOf(getPhoneNumbers)
.filter(relationshipIs 'business')
```

When he wants to use it, he evaluates it and gets the values up to now, filtered:

```
suppliedContacts.evaluate()
```

That technique works for data streams or functions that actually return the values. But when you look at your event-driven program, the functions don’t return the values at all; they invoke a callback with the return value:

```
fs.readFile, 'filename', callback
```

So, how can you extend `withResultOf` to work for asynchronous functions that invoke callbacks?

### 9.4.2. Lazy event handling

Consider again the data for the phone book. This time it’s not returned by a function but instead passed as an argument to a callback:

```
phonebookData = (callback) ->
  callback [
    { name: 'hannibal', number: '555-5551', relationship: 'friend'
}
    { name: 'darth', number: '555-5552', relationship: 'colleague'
}
```